NUMERICAL METHODS IN ENGINEERING, COURSE PROJECT 2

Implementation of the 5-point scheme for the 2D Poisson equation

STEFANO PATITUCCI

MSc Nuclear Engineering, A.Y. 2025/2026

This project focuses on the numerical solution of the two-dimensional Poisson equation on a rectangular domain using the 5-point finite difference scheme. The problem is motivated by the mechanical response of a rectangular sample of biological tissue, such as cartilage or an arterial wall, modeled as an elastic membrane. The elastic behavior of the material is described through a suitable elastic parameter μ, while an imposed force f and prescribed boundary conditions g determine the deformation of the sample. Such an experiment can be very useful for characterizing stress testing of biological samples.

From a numerical point of view, the continuous Poisson problem

$$-\mu\Delta u = f \qquad \text{in } \Omega \tag{1}$$

is discretized on the Cartesian grid (x_i, y_j) leading to a sparse linear system of the form

$$A\mathbf{U} = \mathbf{F}, \tag{2}$$

where A represents the discrete Laplace operator, $\mathbf{U}$ contains the approximate nodal values of the displacement, and $\mathbf{F}$ contains the source term together with the contributions from the boundary conditions.

The main objective of the project is to analyze, implement, and validate the 5-point finite difference scheme for the 2D Poisson problem. After deriving its expected second-order accuracy, the method is implemented in MATLAB and tested on problems with known exact solutions in order to verify the convergence rate. The results are then discussed in terms of accuracy, implementation choices, and agreement with the theoretical error analysis.

Contents

1 Error analysis of the 5-point finite difference scheme **2**
- 1.1 Consistency analysis . . . 2
- 1.2 Stability . . . 3
- 1.3 Convergence . . . 3
- 1.4 Theoretical implications . . . 3

2 MATLAB implementation of the 5-point finite difference scheme **4**

3 Numerical error analysis **5**
- 3.1 Results and conclusions . . . 6

4 Convergence test **7**

5 Implementation challenges **8**
- 5.1 Construction of the sparse matrix . . . 8
- 5.2 Index mapping and solution reshaping . . . 9
- 5.3 Evaluating the exact solution on the discrete grid . . . 9

1 Error analysis of the 5-point finite difference scheme

The error analysis of the 5-point finite difference scheme is based on the equivalence principle: consistency together with stability implies convergence. We consider the Poisson problem with $\mu = 1$,

$$-\Delta u = f \qquad \text{in } \Omega, \tag{3}$$

and assume a uniform Cartesian grid with

$$h_x = h_y = h. \tag{4}$$

At an internal grid point (x_i, y_j), the 5-point finite difference scheme is

$$-\frac{U_{i+1,j} + U_{i-1,j} + U_{i,j+1} + U_{i,j-1} - 4U_{i,j}}{h^2} = f(x_i, y_j) \tag{5}$$

or equivalently,

$$\frac{4U_{i,j} - U_{i+1,j} - U_{i-1,j} - U_{i,j+1} - U_{i,j-1}}{h^2} = f(x_i, y_j). \tag{6}$$

1.1 Consistency analysis

To assess consistency, we insert the exact solution u into the discrete operator and compare it with the exact differential equation. The local truncation error is therefore defined as

$$\mathrm{LTE}_{i,j} = -\frac{u(x_{i+1}, y_j) + u(x_{i-1}, y_j) + u(x_i, y_{j+1}) + u(x_i, y_{j-1}) - 4u(x_i, y_j)}{h^2} - f(x_i, y_j). \tag{7}$$

Using Taylor expansions around (x_i, y_j), we have in the x-direction

$$u(x_{i+1}, y_j) = u(x_i, y_j) + hu_x(x_i, y_j) + \frac{h^2}{2}u_{xx}(x_i, y_j) + \frac{h^3}{6}u_{xxx}(x_i, y_j) + \frac{h^4}{24}u_{xxxx}(\xi_i^+, y_j), \tag{8}$$

$$u(x_{i-1}, y_j) = u(x_i, y_j) - hu_x(x_i, y_j) + \frac{h^2}{2}u_{xx}(x_i, y_j) - \frac{h^3}{6}u_{xxx}(x_i, y_j) + \frac{h^4}{24}u_{xxxx}(\xi_i^-, y_j), \tag{9}$$

with suitable points ξ_i^+ and ξ_i^- between x_i and x_{i+1}, and between x_i and x_{i-1}, respectively. Summing the two expansions gives

$$\frac{u(x_{i+1}, y_j) - 2u(x_i, y_j) + u(x_{i-1}, y_j)}{h^2} = u_{xx}(x_i, y_j) + \frac{h^2}{24}\big[u_{xxxx}(\xi_i^+, y_j) + u_{xxxx}(\xi_i^-, y_j)\big]. \tag{10}$$

Analogously, in the y-direction,

$$\frac{u(x_i, y_{j+1}) - 2u(x_i, y_j) + u(x_i, y_{j-1})}{h^2} = u_{yy}(x_i, y_j) + \frac{h^2}{24}\big[u_{yyyy}(x_i, \eta_j^+) + u_{yyyy}(x_i, \eta_j^-)\big], \tag{11}$$

where η_j^+ and η_j^- are suitable points between y_j and y_{j+1}, and between y_j and y_{j-1}, respectively. Therefore,

$$\begin{aligned} &\frac{u(x_{i+1}, y_j) + u(x_{i-1}, y_j) + u(x_i, y_{j+1}) + u(x_i, y_{j-1}) - 4u(x_i, y_j)}{h^2} = \\ &= -\Delta u(x_i, y_j) + \frac{h^2}{24}\big[u_{xxxx}(\xi_i^+, y_j) + u_{xxxx}(\xi_i^-, y_j) + u_{yyyy}(x_i, \eta_j^+) + u_{yyyy}(x_i, \eta_j^-)\big]. \end{aligned} \tag{12}$$

Since the exact solution satisfies

$$-\Delta u(x_i, y_j) = f(x_i, y_j), \tag{13}$$

it follows that

$$\mathrm{LTE}_{i,j} = -\frac{h^2}{24}\big[u_{xxxx}(\xi_i^+, y_j) + u_{xxxx}(\xi_i^-, y_j) + u_{yyyy}(x_i, \eta_j^+) + u_{yyyy}(x_i, \eta_j^-)\big]. \tag{14}$$

Taking absolute values and using the triangular inequality,

$$|\mathrm{LTE}_{i,j}| \le \frac{h^2}{24}\Big[2\max_{\Omega}|u_{xxxx}| + 2\max_{\Omega}|u_{yyyy}|\Big]. \tag{15}$$

Hence,

$$|\mathrm{LTE}_{i,j}| \leq \frac{h^2}{12}\Big[\max_{\Omega}|u_{xxxx}| + \max_{\Omega}|u_{yyyy}|\Big] \tag{16}$$

therefore, in the maximum norm,

$$\|\mathrm{LTE}\|_\infty \leq Ch^2, \tag{17}$$

where

$$C = \frac{1}{12}\Big[\max_{\Omega}|u_{xxxx}| + \max_{\Omega}|u_{yyyy}|\Big] \tag{18}$$

which shows that the 5-point finite difference scheme is second-order consistent.

1.2 Stability

For the present analysis, stability is assumed: this means that the discrete solution does not amplify perturbations excessively. More precisely, there exists a constant C_s, independent of h, such that

$$\|U\|_\infty \leq C_s\|F\|_\infty. \tag{19}$$

1.3 Convergence

Let

$$e_{i,j} = u(x_i, y_j) - U_{i,j} \tag{20}$$

be the error at the grid point (x_i, y_j). The exact solution inserted into the scheme satisfies the discrete equation up to the local truncation error, while the numerical solution satisfies the finite difference scheme exactly. Subtracting the two equations (5) and (7) gives the discrete error equation

$$-\frac{e_{i+1,j} + e_{i-1,j} + e_{i,j+1} + e_{i,j-1} - 4e_{i,j}}{h^2} = \mathrm{LTE}_{i,j}. \tag{21}$$

Thus, the error satisfies the same discrete operator, with the truncation error acting as right-hand side. Applying the stability estimate (19) to the error equation yields

$$\|e\|_\infty \leq C_s\|\mathrm{LTE}\|_\infty \tag{22}$$

and using the consistency estimate (17),

$$\|\mathrm{LTE}\|_\infty \leq Ch^2, \tag{23}$$

we obtain

$$\|e\|_\infty \leq C_s C h^2. \tag{24}$$

Therefore,

$$\|e\|_\infty = \mathcal{O}\big(h^2\big). \tag{25}$$

Hence, assuming stability, the 5-point finite difference approximation of the 2D Poisson problem with $h_x = h_y = h$ and $\mu = 1$ converges with second-order accuracy.

1.4 Theoretical implications

The result shows that the 5-point finite difference scheme is second-order accurate in space. The reason is that the centered second-order finite difference approximation of each second derivative introduces a truncation error of order h^2. Since the 2D Laplacian is the sum of the second derivatives in the x- and y-directions, the total truncation error remains of order h^2.

By the equivalence principle, consistency and stability imply convergence. Therefore, once stability is assumed, the second-order consistency estimate directly implies second-order convergence of the numerical solution. In practical terms, halving the mesh size h should reduce the error by approximately a factor of 4, provided that the exact solution is sufficiently smooth and round-off errors are negligible.

2 MATLAB implementation of the 5-point finite difference scheme

We implement in MATLAB

```
function [x, y, u] = elliptic_dir_2D(h, g, f, mu);
```

to numerically solve the Poisson problem with Dirichlet boundary conditions

$$-\mu\Delta u = f \quad \text{in } \Omega = (0,1)\times(0,1), \quad u = g \quad \text{on } \partial\Omega \tag{26}$$

with a 5-point stencil based on the centered approximation of the Laplacian with second-order accuracy. For the sake of simplicity, let us consider $h_x = h_y = h$ so that

```
% input
% N : number of subintervals in each direction, h = 1/N
% g : function handle for Dirichlet boundary data, g(x,y)
% f : function handle for source term, f(x,y)
% mu : diffusion / elastic coefficient
%
% output
% x, y : grid points in x and y direction
% u : numerical solution on the full grid
```

We begin with the discretization of the grid

```
M = floor(1/h);
h = 1/M; % enforce exact subdivision of (0,1)

x = linspace(0, 1, M+1)';
y = linspace(0, 1, M+1)';

N = M - 1;
```

and initializing the full solution matrix

```
% rows correspond to y, columns correspond to x
u = zeros(M+1, M+1);
```

The Dirichlet boundary conditions are imposed as

```
u(1, :) = g(x', 0); % bottom boundary, y = 0
u(end, :) = g(x', 1); % top boundary, y = 1
u(:, 1) = g(0, y); % left boundary, x = 0
u(:, end) = g(1, y); % right boundary, x = 1
```

and the costruction of the sparse matrix is done as

```
alpha = 4*mu/h^2; % diagonal entries
beta = -mu/h^2; % off-diagonal entries

e = ones(N,1);
D = spdiags([beta*e, alpha*e, beta*e], [-1, 0, 1], N, N);
E = beta*speye(N);

A = kron(speye(N),D) + kron(spdiags([e,e],[-1,1],N,N),E);
```

The construction of the right-hand side is first done by filling the internal values

```
F = zeros(N,N);

% F internal nodes values
for j = 1:N
    for i = 1:N
        F(j, i) = f(x_int(i), y_int(j));
    end
end
```

with internal grid points

```
x_int = x(2:end-1);
y_int = y(2:end-1);
```

and then by correcting for the boundary conditions

```
F(1, :) = F(1, :) + (mu / h^2) * u(1, 2:end-1); % bottom
F(end, :) = F(end, :) + (mu / h^2) * u(end, 2:end-1); % top
F(:, 1) = F(:, 1) + (mu / h^2) * u(2:end-1, 1); % left
F(:, end) = F(:, end) + (mu / h^2) * u(2:end-1, end); % right
```

this is because, from (6), considering for example a node on the first internal row, its bottom neighbour is $u_{i,0}$ which is known from g, therefore

$$\frac{\mu}{h^2}(4u_{i,1} - u_{i-1,1} - u_{i+1,1} - u_{i,2} - u_{i,0}) = f_{i,1} \tag{27}$$

becomes

$$\frac{\mu}{h^2}(4u_{i,1} - u_{i-1,1} - u_{i+1,1} - u_{i,2}) = f_{i,1} + \frac{\mu}{h^2}u_{i,0}. \tag{28}$$

with boundary values being added to F with a positive sign. To solve the linear system we write

```
U = A\F(:);
```

which outputs $\mathbf{U}$ as a column vector, so we reshape it back into an $N \times N$ matrix

```
u(2:end-1, 2:end-1) = reshape(U, N, N);
```

column-wise, reconstructing the internal solution.

3 Numerical error analysis

We test the implemented scheme with $\mu = 2$, $f = -8$ and Dirichlet boundary coinditions on $\partial\Omega$

$$g(x, y) = x^2 + y^2. \tag{29}$$

The exact solution is

$$u_{\text{ex}}(x, y) = x^2 + y^2 \tag{30}$$

and the numerical error in computed in the maximum norm as

$$E(h) = \max_i \max_j |u_h(x_i, y_j) - u(x_i, y_j)|. \tag{31}$$

We begin by defining parameters and the space discretization steps vector and corresponding errors vector

```
mu = 2;

f = @(x,y) -8 + 0*x + 0*y;
g = @(x,y) x.^2 + y.^2;
uex = @(x,y) x.^2 + y.^2;

h0 = 1/4;
halv = 5;

h_vec = zeros(halv+1,1);
err = zeros(halv+1,1);
```

with chosen initial step $h_0 = 0.25$. Successive halvings and error evaluations are done within the loop

```
for r = 1:halv+1
   h = h0/2^(r-1);
   [x, y, u] = elliptic_dir_2D(h, g, f, mu);

   [X, Y] = ndgrid(x,y);
```

```
    Uex = uex(X,Y); % evaluate exact solution on the full grid

    err(r) = max(max(abs(u - Uex)));
    h_vec(r) = h;
end
```

which iteratively

- calculates $h_r = h_0/2^{r-1}$ and fills the corresponding vector `h_vec`;
- calculates the numerical solution matrix `u` using the function `elliptic_dir_2D`;
- creates the corresponding discretized grid `[X, Y] = ndgrid(x,y)` at every halfing of the space step and evalutes the exact solution `uex` on that grid;
- computes the error vector `err(r)`.

The convergence order is estimated as usual as

```
p = log2(err(1:end-1)./err(2:end));
```

3.1 Results and conclusions

Results are reported in the following Table 1 and corresponding log-log plot in Figure 3.1.

h	error	estimated order p
0.25	1.1102e-16	-
0.125	1.3323e-15	-3.585
0.0625	4.2188e-15	-1.663
0.03125	1.4322e-14	-1.7633
0.015625	4.0523e-14	-1.5005
0.0078125	2.4281e-13	-2.583

Table 1 Infinity-norm error and estimated convergence order for successive halvings of the mesh size h.

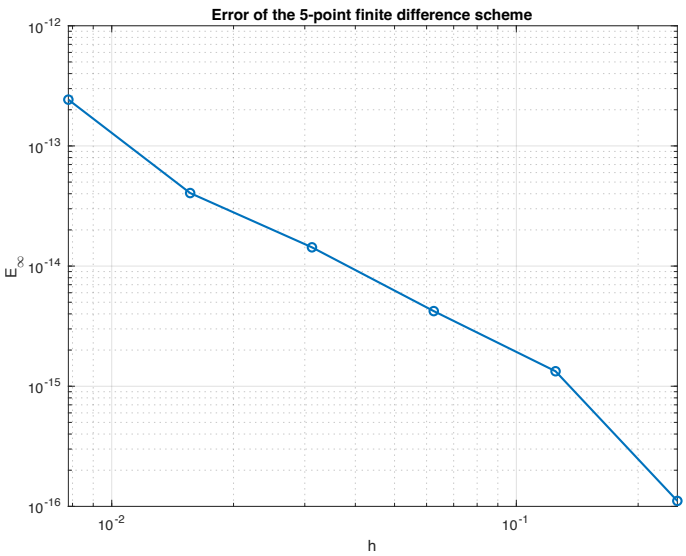

Figure 3.1 Log-log plot of the infinity-norm error E_∞ versus the mesh size h. The error remains at machine precision because the exact solution is quadratic, so the 5-point Laplacian introduces no discretization error.

We can conclude that this test does not measure convergence order of the 5-point scheme. Errors are all around machine precision of order

$$10^{-16}\text{–}10^{-13} \tag{32}$$

meaning that the numerical solution is essentially exact. The negative estimated orders of convergence do not mean the method is divergent: the only mean that error is dominated by floating-point round-off and by the conditioning of the linear system, which also explains the decreasing trend for larger h.

The reason for this is that the exact solution $u(x, y) = x^2 + y^2$ is quadratic, and the 5-point finite difference Laplacian is exact on quadratic polynomials. To verify the $\mathcal{O}(h^2)$ convergence rate, a non-quadratic manufactured solution should be used.

4 Convergence test

To actually prove the second-oder convergence of the implemented method, we choose a non-polynomial exact solution of the form

$$u_{\text{ex}}(x, y) = \sin(\pi x)\sin(\pi y), \qquad (x, y) \in \Omega \tag{33}$$

with $g(x, y) = u_{\text{ex}}(x, y) = 0$ on $\partial\Omega$ since the sine vanishes on the boundary. Computing the laplacian yields

$$\Delta u_{\text{ex}} = \frac{\partial^2 u_{\text{ex}}}{\partial x^2} + \frac{\partial^2 u_{\text{ex}}}{\partial y^2} = -2\pi^2 \sin(\pi x)\sin(\pi y) \tag{34}$$

therefore

$$f(x, y) = -\mu\Delta u_{\text{ex}} = 2\mu\pi^2 \sin(\pi x)\sin(\pi y). \tag{35}$$

Running again the same method now gives the following results.

h	error	estimated order p
0.25	0.053029	-
0.125	0.012951	2.0338
0.0625	0.003219	2.0084
0.03125	0.00080358	2.0021
0.015625	0.00020082	2.0005
0.0078125	5.0201e-05	2.0001

Table 2 Infinity-norm error and estimated convergence order for successive halvings of the mesh size h.

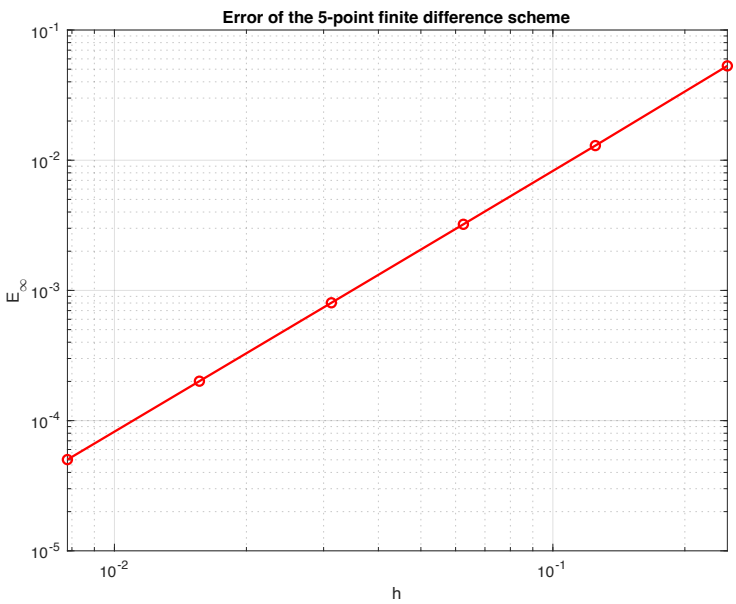

Figure 4.1 Log-log plot of the infinity-norm error E_∞ versus the mesh size h for the 5-point finite difference scheme, showing the expected linear trend associated with second-order convergence.

The numerical results now confirm the expected second-order convergence of the 5-point finite difference scheme: as the mesh size is halved, the infinity-norm error decreases by approximately a factor of 4, and the estimated convergence order p approaches 2, in agreement with the theoretical truncation error of the centered finite difference approximation of the Laplacian of order $\mathcal{O}(h^2)$.

5 Implementation challenges

Some discussion of the code used has already been discussed in the dedicated implementation section. Here, we therefore focus only on the main challenges encountered while developing and testing the code.

5.1 Construction of the sparse matrix

The matrix A is built using a block structure: $D \in \mathbb{R}^{N\times N}$ is the block that handles coupling withing the same row, while $E \in \mathbb{R}^{N\times N}$ accounts for coupling between adjecent rows.

We begin by creating `e = ones(N,1)` a column vector of length N filled with 1s, usefull to build diagonal vectors of the same size. The matrix D in constructed as

```
D = spdiags([beta*e, alpha*e, beta*e], [-1, 0, 1], N, N);
```

where `spdiags(B,d,m,n)` creates a sparse $m \times n$ matrix whose diagonals are specified by the columns of `B` and whose positions are specified by `d` (with respect to 0 being the diagonal). The matrix $E = \beta\mathbf{I}$ is just

```
E = beta*speye(N);
```

where `speye(N)` creates the $N \times N$ sparse identity matrix. The full 2D matrix is then assembled through Kronecker products

```
A = kron(speye(N),D) + kron(spdiags([e,e],[-1,1],N,N),E);
```

The first term `kron(speye(N),D)` builds a block diagonal matrix with N copies of D along the diagonal

$$\begin{bmatrix} D & & \\ & D & \\ & & \ddots \end{bmatrix} \tag{36}$$

while the second term `kron(spdiags([e,e],[-1,1],N,N),E)` adds the coupling between adjacent rows. The matrix `spdiags([e,e],[-1,1],N,N)` creates an $N\times N$ sparse matrix with ones on the lower and upper diagonals

$$\begin{bmatrix} 0 & 1 & & \\ 1 & 0 & 1 & \\ & 1 & 0 & \ddots \\ & & \ddots & \ddots \end{bmatrix} \tag{37}$$

so that taking its Kronecker product with E places the block E above and below the main block diagonal. The sum of these two terms yields the final block tridiagonal form

$$A = \begin{bmatrix} D & E & & \\ E & D & E & \\ & E & D & \ddots \\ & & \ddots & \ddots \end{bmatrix}. \tag{38}$$

5.2 Index mapping and solution reshaping

The finite difference scheme is naturally written on the internal grid nodes, where each unknown is associated with a pair of indices (i, j). However, the algebraic system

$$A\mathbf{U} = \mathbf{F} \tag{39}$$

requires the unknowns to be stored in a column vector. Therefore, each internal node must be associated with a single linear index k. This is the purpose of the index map: it allows us to pass from the matrix representation of the numerical solution to the vector representation required by the linear solver.

In MATLAB, this mapping is handled naturally by column-wise vectorization. The backslash operator in

```
U = A\F(:);
```

solves the linear system $A\mathbf{U} = \mathbf{f}$ for the internal nodes without computing the inverse of A, while the expression `F(:)` implicitly converts the matrix $\mathbf{F}$ into a column vector $\mathbf{f}$ with column-wise mapping, that is

$$(i, j) \mapsto k, \qquad k = (i-1)N + j. \tag{40}$$

After solving the linear system, the output $\mathbf{U}$ is a column vector containing only the values at the internal nodes. However, for visualization, error computation, and comparison with the exact solution, it is more convenient to work again with a two-dimensional grid function. For this reason, the numerical solution vector must be reshaped into an $N \times N$ matrix

```
u(2:end-1, 2:end-1) = reshape(U, N, N);
```

where `reshape(U,N,N)` takes the entries of the vector `U` and fills an $N \times N$ matrix column-wise, following the same ordering used by `F(:)`: thus, reshape is the inverse operation of the previous vectorization, provided that the dimensions are consistent. Finally

```
u(2:end-1, 2:end-1)
```

selects only the internal part of the full solution matrix `u`. The first and last rows, and the first and last columns, correspond to boundary nodes and have already been assigned through the Dirichlet boundary condition. Hence, the reshaped matrix is inserted only in the internal block of `u`, reconstructing the full numerical solution on the grid.

5.3 Evaluating the exact solution on the discrete grid

The last implementation issue that needs to be addressed concerne the evaluation of the solution for the Poisson problem on the same discrete grid used by the numerical scheme. Since the error is computed though

$$\max_{i,j} |u_{i,j} - u_{\text{ex}}(x_i, y_j)| \tag{41}$$

the numerical solution and the exact solution must be stored as matrices with the same size and the same indexing convention (x_i, y_j). This is achieved by

```
[X, Y] = ndgrid(x,y);
Uex = uex(X,Y);
```

The command `ndgrid(x,y)` builds two coordinate matrices `X` and `Y` from the one-dimensional vectors `x` and `y`. More precisely, it returns matrices such that

$$X_{ij} = x_i, \qquad Y_{ij} = y_j. \tag{42}$$

so that each entry (i, j) of the pair `(X,Y)` corresponds exactly to the physical grid point (x_i, y_j). When the exact solution is then evaluated as

```
Uex = uex(X,Y);
```

MATLAB applies the function handle `uex` element-wise to the matrices `X` and `Y`.

For example, if `uex = @(x,y)x.^2 + y.^2` then `Uex = uex(X,Y)` produces the matrix of element

$$\texttt{Uex}_{ij} = X_{ij}^2 + Y_{ij}^2 = x_i^2 + y_j^2. \tag{43}$$

This matrix form is necessary because the numerical solution `u` is also a matrix: the error can thus be computed entry by entry through

```
err(r) = max(max(abs(u - Uex)));
```

with the first `max` taking the maximum along one matrix direction, producing a row vector, while the second takes the maximum of that vector, giving the largest absolute error over the whole grid.

The use of `ndgrid(x,y)` is important because it matches the storage convention used in the scheme. In this implementation,

$$\texttt{u}_{ij} \approx u(x_i, y_j), \tag{44}$$

so the first index of the matrix corresponds to the x-coordinate and the second index corresponds to the y-coordinate, which is exactly the convention produced by `ndgrid`. By contrast, `meshgrid(x,y)` uses the opposite convention

$$X_{ij} = x_j, \qquad Y_{ij} = y_i. \tag{45}$$

constructing matrices where rows correspond to y-values and columns correspond to x-values. This is useful for many plotting routines, but it is not consistent with the way the solution matrix `u` is stored in this finite difference implementation.